

WakeBuddy 개발 방식별 코드 비교 분석

1. 비교 대상

본 회의에서는 동일한 공유 알람 애플리케이션인 WakeBuddy를 대상으로, 개발 방식에 따라 생성된 두 코드의 차이를 비교하였다.

첫 번째 코드는 **요구사항 분석 → 설계 → 구현 → 테스트 → 품질관리** 절차를 거쳐 작성된 코드이다. 이 코드는 Firebase, Firestore, 서비스 계층, 모델, 유틸리티, 테스트 코드가 분리되어 있으며, 실제 서비스 구조에 가깝게 구성되어 있다.

두 번째 코드는 별도의 요구사항 분석이나 설계 문서 작성 없이 **AI에게 바로 요청하여 생성한 코드**이다. 이 코드는 빠르게 동작 가능한 형태로 구성되어 있다.

2. 전체 구조 차이

첫 번째 코드는 기능별 책임이 비교적 명확하게 나뉘어 있다. 예를 들어 `AlarmService`, `AuthService`, `FriendService`, `FriendRequestService`, `NotificationService`, `PermissionService` 처럼 서비스 계층이 세분화되어 있으며, 각 서비스가 담당하는 기능이 분리되어 있다. 또한 `AlarmDateUtils`, `AuthValidationUtils`, `alarmSchedule` 처럼 공통 로직을 유틸리티로 분리하여 재사용할 수 있도록 구성되어 있다.

반면 두 번째 코드는 빠른 구현을 위해 구조가 단순하다. `alarmService`, `authService`, `friendService`, `notificationService`, `storageService` 정도로 나누어져 있지만, 전체적으로 화면과 서비스가 간단히 연결되는 형태이다. 기능 구현은 가능하지만, 요구사항 변경이나 기능 확장 시 구조적으로 한계가 발생할 가능성이 있다.

구분	첫 번째 코드	두 번째 코드
개발 방식	프로세스 기반 개발	AI 직접 생성
구조	서비스, 모델, 유틸, 테스트 분리	화면과 간단한 서비스 중심
테스트	Jest 테스트 포함	테스트 코드 거의 없음
품질관리	입력 검증, 권한 검증, 알림 검증, 테스트 수행	기본 동작 중심
확장성	높음	낮음
실제 서비스 적합성	비교적 높음	프로토타입 수준

3. 요구사항 분석 반영 차이

첫 번째 코드는 요구사항이 코드에 구체적으로 반영되어 있다. 공유 알람 앱의 핵심 요구사항인 **본인 알람 관리, 친구 알람 관리, 친구 요청, 친구 수락 후 권한 부여, 알람 생성/수정/삭제 권한** 등이 서비스 단위로 분리되어 있다.

특히 `PermissionService` 가 존재한다는 점이 중요하다. 이 서비스는 현재 사용자가 특정 대상에게 알람을 생성할 수 있는지, 알람을 조회·수정·삭제할 수 있는지를 판단한다. 즉, 요구사항 분석 단계에서 도출된 “친구 관계가 성립된 경우에만 상대방 알람을 관리할 수 있다”는 조건이 코드에 명확히 반영되어 있다.

반면 두 번째 코드는 친구 요청, 친구 목록, 친구 알람 조회 기능이 있기는 하지만, 권한 검증이 별도의 독립된 계층으로 분리되어 있지 않다. 예를 들어 친구에게 알람을 만들 수 있는지에 대한 판단이 전체 구조에서 명확한 정책으로 관리되기보다는 각 기능 내부에서 단순히 처리된다. 따라서 요구사항이 복잡해질 경우 누락되거나 중복 구현될 가능성이 있다.

4. 설계 측면 차이

첫 번째 코드는 설계 원리에 더 가깝게 구성되어 있다. 기능별 서비스가 분리되어 있어 단일 책임 원칙을 비교적 잘 따른다. 예를 들어 인증은 `AuthService`, 알람 관리는 `AlarmService`, 친구 요청은 `FriendRequestService`, 친구 관계는 `FriendService`, 알람 예약은 `NotificationService`, 권한 판단은 `PermissionService` 가 담당한다.

또한 모델도 별도로 분리되어 있다. `Alarm`, `User`, `Friendship`, `FriendRequest` 와 같은 타입이 정의되어 있어 데이터 구조가 명확하다. 이 때문에 기능을 수정하거나 확장할 때 어떤 데이터가 사용되는지 파악하기 쉽다.

두 번째 코드는 설계보다는 구현 속도에 초점이 맞춰져 있다. 타입 정의가 `index.ts` 에 모여 있고, 저장소 역할을 하는 `storageService` 가 사용자, 알람, 친구 요청, 친구 목록을 모두 처리한다. 이는 작은 규모에서는 편하지만, 기능이 많아질수록 하나의 파일에 책임이 몰리는 문제가 발생할 수 있다.

설계 요소	첫 번째 코드	두 번째 코드
단일 책임	서비스별 책임 분리	일부 서비스에 책임 집중
데이터 모델	모델 파일로 분리	타입 정의가 한곳에 집중
권한 설계	<code>PermissionService</code> 로 명확히 분리	기능 내부에서 단순 처리
유지보수성	높음	기능 증가 시 낮아질 가능성

5. 구현 방식 차이

첫 번째 코드는 실제 요구사항을 단계적으로 구현한 형태이다. 기능을 바로 화면에 연결하기보다, **서비스 계층과 유틸리티 계층을 먼저 나누고 각 기능이 어떤 책임을 가지는지 구분**하였

다. 이로 인해 코드의 흐름이 비교적 명확하며, 특정 기능을 수정할 때 영향을 받는 범위를 예측하기 쉽다.

예를 들어 알람 기능은 단순히 화면에서 바로 처리되는 것이 아니라, 알람 관리 로직, 알람 예약 로직, 날짜 계산 로직 등이 분리되어 있다. 인증 관련 기능도 입력값 검증 로직이 별도로 분리되어 있어 화면 코드와 핵심 로직이 섞이지 않는다. 또한 친구 기능 역시 친구 요청, 친구 관계, 권한 판단이 각각 나뉘어 있어 기능의 의미가 명확하다.

반면 두 번째 코드는 화면 구성과 기능 동작이 빠르게 연결되어 있어 짧은 시간 안에 앱의 전체 흐름을 확인하기 좋다. 그러나 기능이 추가되거나 요구사항이 변경되면 기존 구조를 다시 정리해야 할 가능성이 높다.

예를 들어 두 번째 코드는 알람, 친구, 인증 기능이 구현되어 있지만, 일부 로직이 화면 흐름이나 하나의 서비스 내부에 함께 포함되는 경향이 있다. 이 경우 처음에는 이해하기 쉽지만, 기능이 늘어나면 코드의 책임 범위가 불분명해질 수 있다. 또한 같은 검증 로직이나 조건 판단이 여러 곳에 반복될 가능성이 있어 유지보수 시 수정 누락이 발생할 수 있다.

6. 알람 기능 차이

첫 번째 코드는 알람 생성 시 입력값 검증, 알람 예약, 반복 알람 처리, 알람 ID 저장 등이 비교적 체계적으로 구현되어 있다. `NotificationService` 는 Android 알람 채널, 커스텀 사운드, 반복 없음/매일/요일 반복 알람을 구분해서 처리한다. 또한 알람 ID를 저장하고 취소할 수 있도록 구성되어 있어 알람 수정이나 삭제 시 기존 알람을 정리할 수 있다.

두 번째 코드도 알람 예약 기능은 존재한다. 하지만 구현 방식이 단순하다. 반복 없음 알람은 현재 시간이 지났으면 다음 날로 넘기고, 반복 알람은 요일별로 예약한다. 기본적인 동작은 가능하지만, 커스텀 사운드나 플랫폼별 세부 설정, 알람 ID 관리, 실패 처리 등이 첫 번째 코드보다 단순하다.

7. 친구 기능 차이

첫 번째 코드는 친구 요청과 친구 관계를 구분한다. `FriendRequestService` 는 친구 요청 생성, 중복 요청 방지, 요청 수락/거절 등을 담당하고, `FriendService` 는 실제 친구 관계 조회와 생성, 친구 여부 확인을 담당한다. 이처럼 요청 상태와 친구 관계를 분리하면 기능의 의미가 명확해지고, 추후 요청 취소, 차단, 친구 삭제 같은 기능을 추가하기 쉽다.

두 번째 코드는 `friendService` 에서 친구 요청 전송, 수락, 거절, 친구 목록 조회를 한 번에 처리한다. 구조가 간단해서 이해하기 쉽지만, 기능이 많아지면 파일 하나가 여러 책임을 갖게 된다. 또한 친구 관계나 요청 상태에 대한 검증 로직이 첫 번째 코드보다 단순하다.

8. 테스트 차이

첫 번째 코드에는 테스트 코드가 포함되어 있다. 대표적으로 `AlarmDateUtils.test.ts`, `AuthValidationUtils.test.ts`, `PermissionService.test.ts` 가 있다. 이 테스트들은 알람 시간 계산, 로그인/회원가입 입력값 검증, 친구 권한 검증 등 핵심 로직을 검증한다.

예를 들어 알람 시간이 현재 시간보다 이전이면 오류가 발생하는지, 공백 이메일이나 비밀번호를 입력하면 오류가 발생하는지, 친구가 아닌 사용자에게 알람 생성 권한이 없는지 등을 테스트한다. 이는 단순히 앱이 실행되는지 확인하는 수준을 넘어서, 요구사항이 실제 코드에서 지켜지는지 검증하는 과정이다.

반면 두 번째 코드에는 별도의 테스트 코드가 확인되지 않는다. 따라서 기능이 정상적으로 동작하는지는 직접 앱을 실행해 수동으로 확인해야 한다. 테스트가 없으면 수정 후 기존 기능이 깨졌는지 확인하기 어렵고, 기능이 많아질수록 품질 관리가 어려워진다.

테스트 항목	첫 번째 코드	두 번째 코드
알람 시간 계산 테스트	있음	없음
로그인/회원가입 입력 검증 테스트	있음	없음
권한 검증 테스트	있음	없음
자동 테스트 실행 환경	Jest 기반	명확하지 않음
회귀 테스트 가능성	높음	낮음

9. 품질관리 차이

첫 번째 코드는 입력값 검증, 권한 검증, 예외 처리, 테스트 코드, 주석, 서비스 분리, 타입 정의 등이 포함되어 있다. 특히 코드에 주석이 많아 각 함수의 목적과 동작을 이해하기 쉽고, 유지보수 시 어떤 부분을 수정해야 하는지 파악하기 쉽다.

두 번째 코드는 품질관리 요소가 상대적으로 부족하다. 예외 처리는 일부 존재하지만, 입력 검증이나 권한 검증이 체계적으로 분리되어 있지 않고, 테스트 코드도 부족하다.

10. 유지보수성 차이

첫 번째 코드는 파일 수와 코드량이 더 많지만, 책임이 분리되어 있어 유지보수성이 높다. 특정 기능을 수정할 때 관련 서비스나 유틸리티만 확인하면 되기 때문이다. 예를 들어 알람 시간 계산 방식만 수정하려면 `AlarmDateUtils` 또는 `alarmSchedule` 을 보면 되고, 권한 정책을 바꾸려면 `PermissionService` 를 수정하면 된다.

두 번째 코드는 코드량이 적고 단순해서 처음 이해하기는 쉽다. 하지만 기능이 추가될수록 `storageService`, `friendService`, `alarmService` 등에 로직이 계속 누적될 가능성이 크다. 이 경우 특정 기능 변경이 다른 기능에 영향을 줄 수 있고, 테스트가 없기 때문에 수정 후 안정성을 보장하기 어렵다.

11. 코드 규모 차이

첫 번째 코드는 서비스, 모델, 유틸리티, 테스트 코드가 포함되어 있어 전체 코드 규모가 더 크다. 이는 단순히 코드가 많다는 의미가 아니라, 요구사항 분석과 설계를 통해 필요한 책임을 나누고 검증 코드를 추가했기 때문이다.

두 번째 코드는 전체적으로 더 짧고 단순하다. 빠르게 결과물을 만들기에는 좋지만, 실제 서비스 수준의 품질이나 확장성을 확보하기에는 부족하다.

구분	첫 번째 코드	두 번째 코드
주요 서비스/유틸/테스트 코드량	약 2,200줄 이상	약 800줄 내외
테스트 코드	포함	거의 없음
모델 분리	있음	단순 타입 중심
구현 복잡도	높음	낮음
품질 확보 가능성	높음	낮음

13. 장단점 정리

첫 번째 코드의 장점

첫 번째 코드는 요구사항 분석과 설계 과정을 거쳤기 때문에 전체 구조가 명확하다. 기능별 책임이 분리되어 있고, 권한 검증과 입력 검증이 독립적으로 관리된다. 또한 테스트 코드가 포함되어 있어 기능이 요구사항대로 동작하는지 검증할 수 있다. 유지보수와 기능 확장에도 유리하다.

첫 번째 코드의 단점

구조가 상대적으로 복잡하고 파일 수가 많기 때문에 초기에 이해하는 데 시간이 더 걸린다. 서비스, 모델, 유틸리티, 테스트 파일이 나뉘어 있어 단순한 프로토타입을 빠르게 만들기에는 부담이 있을 수 있다.

두 번째 코드의 장점

두 번째 코드는 구조가 단순하고 빠르게 실행 가능한 형태로 작성되어 있다. 화면, 로그인, 친구, 알림, 알림 기능이 한 번에 구성되어 있어 프로토타입 제작에는 적합하다. 개발 초기 아이디어 검증이나 화면 흐름 확인에는 유용하다.

두 번째 코드의 단점

요구사항 분석과 설계가 부족하기 때문에 기능 간 책임 분리가 약하다. 테스트 코드가 부족해 기능 수정 후 안정성을 검증하기 어렵고, 권한 검증도 첫 번째 코드보다 명확하지 않다. 기

능이 많아질수록 유지보수성이 떨어질 가능성이 있다.

14. 최종 비교 결론

두 코드의 가장 큰 차이는 **기능 구현 여부가 아니라 개발 과정에서 품질을 어떻게 확보했는가**이다.

두 번째 코드는 AI에게 바로 요청하여 빠르게 앱의 기본 형태를 만들 수 있다는 장점이 있다. 하지만 요구사항 분석, 설계, 테스트, 품질검증 과정이 생략되어 실제 서비스에 필요한 권한 관리, 데이터 공유, 보안, 유지보수성, 테스트 가능성이 부족하다.

반면 첫 번째 코드는 개발 속도는 느릴 수 있지만, 요구사항을 분석하고 설계한 뒤 구현했기 때문에 구조가 명확하고 안정성이 높다. 특히 친구 관계 기반 권한 검증, Firebase 기반 데이터 관리, 알람 시간 계산 유틸리티, 입력값 검증, 단위 테스트가 포함되어 있어 강의록에서 다룬 소프트웨어 공학 프로세스의 효과를 보여준다.

따라서 본 비교를 통해 **AI를 활용하면 빠른 구현은 가능하지만, 요구사항 분석·설계·테스트·품질관리 과정을 거친 코드가 실제 서비스 품질과 유지보수성 측면에서 더 우수함을 확인할 수 있었다.**